

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Shaik Nuha	Reg. No.:	192472242
Section / Batch:	CSE-AI	Date:	

Instructions to Students

- Answer both questions carefully. Each question carries 50 marks. Total: 100 marks.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Description	Questions	Total Marks
Comparative Analysis	Focus on Comparison of different Algorithms using dynamic programming and greedy strategies.	Q1 – Q2	100

SECTION A – Comparative Analysis

Q103. Assembly Line Scheduling vs Optimal BST (DP Decision Comparison)

Compare Assembly Line Scheduling and Optimal BST problems using DP. Analyze differences in decision variables and cost functions. Evaluate recurrence formulation in both problems. Discuss computational complexity. Examine scalability. Generate insights on structured decision optimization.

Q104. Travelling Salesperson vs Knapsack (Complexity Comparison)

Compare TSP and Knapsack problems solved using DP. Analyze exponential vs polynomial complexity behaviour. Evaluate scalability limitations. Discuss memory requirements. Examine applicability in real systems. Generate insights on feasibility of DP solutions.

Code of Conduct

I Shaik Nuha (192472242) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____Sk.Nuha_____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Mark Evaluation:

Questions	Selection of Studies/Parameters (20)	Comparison & Differentiation (25)	Critical Evaluation (25)	Synthesis & Insight Generation (20)	Clarity (10)	Total Marks (Each - 50)
Q1						
Q2						
Overall Total						

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q103.

Aim

To compare Assembly Line Scheduling and Optimal Binary Search Tree (OBST) problems using Dynamic Programming and analyze their decision-making strategies, cost functions, complexity, and scalability.

Problem Statement

Assembly Line Scheduling aims to minimize manufacturing time by choosing the best assembly line at each station. Optimal BST aims to minimize search cost by selecting the best root node arrangement based on access frequencies. Both use Dynamic Programming but differ in their decision variables and optimization objectives.

Algorithm:

Assembly Line Scheduling

1. Initialize entry times for both assembly lines.
2. For each station:
 - Calculate cost of staying on the same line.
 - Calculate cost of switching lines.
 - Choose minimum cost.
3. Add exit time.
4. Return minimum total production time.
- 5.

Optimal BST

1. Store frequencies of keys.
2. Consider every key as a possible root.
3. Compute left and right subtree costs.
4. Select root with minimum total cost.
5. Store results in DP table.
6. Return minimum search cost.

Comparison:

Feature	Assembly Line Scheduling	Optimal BST
Decision Variable	Stay/Switch Line	Select Root Node
Objective	Minimize Production Time	Minimize Search Cost
Structure	Sequential	Hierarchical
Time Complexity	$O(n)$	$O(n^3)$
Space Complexity	$O(n)$	$O(n^2)$

Python Code:

Assembly Line Scheduling

```
def assembly_line(a1, a2):
    n = len(a1)
    f1 = [0] * n
    f2 = [0] * n
    f1[0] = a1[0]
    f2[0] = a2[0]
    for i in range(1, n):
        f1[i] = min(f1[i-1], f2[i-1]) + a1[i]
        f2[i] = min(f2[i-1], f1[i-1]) + a2[i]
    return min(f1[n-1], f2[n-1])

a1 = [4, 5, 3, 2]
a2 = [2, 10, 1, 4]
print("Minimum Time =", assembly_line(a1, a2))
```

Sample Output:

```
==== RESTART: C:\Users\nuhas\AppData\Local\Programs\Python\Python312\co4.py ====
Minimum Time = 10
```

Optimal BST Cost Calculation

```
def optimal_bst(freq):
    n = len(freq)
    dp = [[0]*n for _ in range(n)]
    for i in range(n):
        dp[i][i] = freq[i]
    for length in range(2, n+1):
        for i in range(n-length+1):
            j = i + length - 1
            dp[i][j] = float('inf')
            total = sum(freq[i:j+1])
            for r in range(i, j+1):
                left = dp[i][r-1] if r > i else 0
                right = dp[r+1][j] if r < j else 0
                dp[i][j] = min(dp[i][j],
                               total + left + right)
    return dp[0][n-1]

freq = [3, 2, 4]
print("Optimal BST Cost =", optimal_bst(freq))
```

Sample Output:

```
==== RESTART: C:\Users\nuhas\AppData\Local\Programs\Python\Python312\co4.py ====  
Optimal BST Cost = 16  
> |
```

Conclusion

Assembly Line Scheduling focuses on sequential production decisions and achieves linear complexity. OBST focuses on hierarchical search optimization and requires cubic complexity. Both demonstrate how Dynamic Programming efficiently solves structured optimization problems.

Q104:

Aim

To compare Travelling Salesperson Problem (TSP) and Knapsack Problem using Dynamic Programming and analyze their complexity, memory usage, scalability, and practical applicability.

Problem Statement

TSP finds the shortest route that visits every city exactly once and returns to the starting city. Knapsack maximizes profit while keeping weight within a capacity limit. Both use Dynamic Programming but differ significantly in complexity and scalability.

Algorithm:

Travelling Salesperson Problem

1. Start from a source city.
2. Maintain visited city states.
3. Compute minimum cost for all possible city subsets.
4. Store results using DP and bit masking.
5. Return minimum tour cost.

Knapsack Problem

1. Initialize DP table.
2. For each item:
 - Include the item if capacity allows.
 - Exclude the item.
3. Store maximum profit.
4. Return optimal value.

Comparison:

Feature	TSP	Knapsack
Goal	Minimum Route Cost	Maximum Profit
DP State	City + Visited Set	Item + Capacity
Time Complexity	$O(n^2 \times 2^n)$	$O(nW)$
Space Complexity	$O(n \times 2^n)$	$O(nW)$
Scalability	Limited	Good
Complexity Type	Exponential	Polynomial/Pseudo-Polynomial

Python Code:

Travelling Salesperson Problem

```
from itertools import permutations
```

```
def tsp(graph):
    n = len(graph)
    min_cost = float('inf')
    for path in permutations(range(1, n)):
        cost = graph[0][path[0]]
        for i in range(len(path)-1):
            cost += graph[path[i]][path[i+1]]
        cost += graph[path[-1]][0]
        min_cost = min(min_cost, cost)
    return min_cost

graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]

print("Minimum Tour Cost =", tsp(graph))
```

Sample Output:

```
>>> |
    | = RESTART: C:\Users\nuhas\AppData\Local\Programs\Python\Python312\co4.py
    | Minimum Tour Cost = 80
>>> |
```

0/1 Knapsack

```
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            if wt[i-1] <= w:
                dp[i][w] = max(
                    val[i-1] + dp[i-1][w-wt[i-1]],
                    dp[i-1][w]
                )
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]
wt = [10,20,30]
val = [60,100,120]
W = 50
print("Maximum Profit =", knapsack(wt, val, W))
```

Sample Output:

```
>>> | type "help", "copyright", "credits" or "license()" for more information.
= RESTART: C:\Users\nuhas\AppData\Local\Programs\Python\Python312\co4.py
Maximum Profit = 220
```

Conclusion

TSP suffers from exponential growth in time and memory requirements, making it difficult to scale for large inputs. Knapsack has much better scalability and is widely used in resource allocation and optimization systems. Dynamic Programming is highly practical for Knapsack but becomes computationally expensive for large TSP instances.